

# Mathematics of Machine Learning

A Visual Introduction to the Core Math Behind ML

Your Name

September 3, 2026

# Contents

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>1</b> | <b>Linear Algebra Foundations</b>            | <b>2</b>  |
| 1.1      | Vectors and Vector Operations . . . . .      | 2         |
| 1.2      | Matrices and Matrix Multiplication . . . . . | 2         |
| 1.3      | Eigenvalues and Eigenvectors . . . . .       | 2         |
| 1.4      | Why This Matters for ML . . . . .            | 3         |
| <b>2</b> | <b>Probability and Statistics</b>            | <b>5</b>  |
| 2.1      | Random Variables and Distributions . . . . . | 5         |
| 2.2      | Bayes' Theorem . . . . .                     | 5         |
| 2.3      | Maximum Likelihood Estimation . . . . .      | 6         |
| 2.4      | Cross-Entropy and KL Divergence . . . . .    | 6         |
| <b>3</b> | <b>Calculus and Optimization</b>             | <b>8</b>  |
| 3.1      | Derivatives and Gradients . . . . .          | 8         |
| 3.2      | The Chain Rule . . . . .                     | 8         |
| 3.3      | Gradient Descent . . . . .                   | 8         |
| 3.4      | Variants of Gradient Descent . . . . .       | 10        |
| 3.5      | Convexity . . . . .                          | 10        |
| <b>4</b> | <b>Regression and Classification</b>         | <b>11</b> |
| 4.1      | Linear Regression . . . . .                  | 11        |
| 4.2      | Logistic Regression . . . . .                | 11        |
| 4.3      | Regularization . . . . .                     | 13        |
| <b>5</b> | <b>Neural Networks and Backpropagation</b>   | <b>14</b> |
| 5.1      | The Forward Pass . . . . .                   | 14        |
| 5.2      | Activation Functions . . . . .               | 14        |
| 5.3      | The Loss Function . . . . .                  | 15        |
| 5.4      | Backpropagation . . . . .                    | 15        |
| 5.5      | Putting It All Together . . . . .            | 16        |

# Chapter 1

## Linear Algebra Foundations

Linear algebra is the language machine learning is written in: data points are vectors, datasets are matrices, and models are functions built from matrix operations.

### 1.1 Vectors and Vector Operations

A vector  $\mathbf{x} \in \mathbb{R}^n$  is an ordered list of numbers  $\mathbf{x} = (x_1, x_2, \dots, x_n)$ . In ML, a single data point (e.g. a row of features) is almost always represented as a vector.

**Definition 1.1** (Dot Product). For  $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ ,

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta,$$

where  $\theta$  is the angle between the vectors. The dot product measures how much two vectors point in the same direction, and underlies operations like computing a weighted sum  $\mathbf{w} \cdot \mathbf{x}$  in a linear model.

**Definition 1.2** (Vector Projection). The projection of  $\mathbf{a}$  onto  $\mathbf{b}$  is

$$\text{proj}_{\mathbf{b}} \mathbf{a} = \frac{\mathbf{a} \cdot \mathbf{b}}{\mathbf{b} \cdot \mathbf{b}} \mathbf{b}.$$

### 1.2 Matrices and Matrix Multiplication

A matrix  $A \in \mathbb{R}^{m \times n}$  maps vectors in  $\mathbb{R}^n$  to vectors in  $\mathbb{R}^m$  via  $\mathbf{y} = A\mathbf{x}$ . Every layer of a neural network, at its core, is a matrix multiplication followed by a nonlinearity.

$$(A\mathbf{x})_i = \sum_{j=1}^n A_{ij} x_j$$

**Key Idea.** Matrix multiplication is function composition. Stacking layers  $\mathbf{h} = A_2 \sigma(A_1 \mathbf{x})$  is how deep networks build up complex transformations from simple linear pieces plus nonlinearities.

### 1.3 Eigenvalues and Eigenvectors

**Definition 1.3** (Eigenvector). A nonzero vector  $\mathbf{v}$  is an eigenvector of a square matrix  $A$  with eigenvalue  $\lambda$  if

$$A\mathbf{v} = \lambda\mathbf{v}.$$

That is,  $A$  only stretches or shrinks  $\mathbf{v}$  — it doesn't rotate it off its own line.

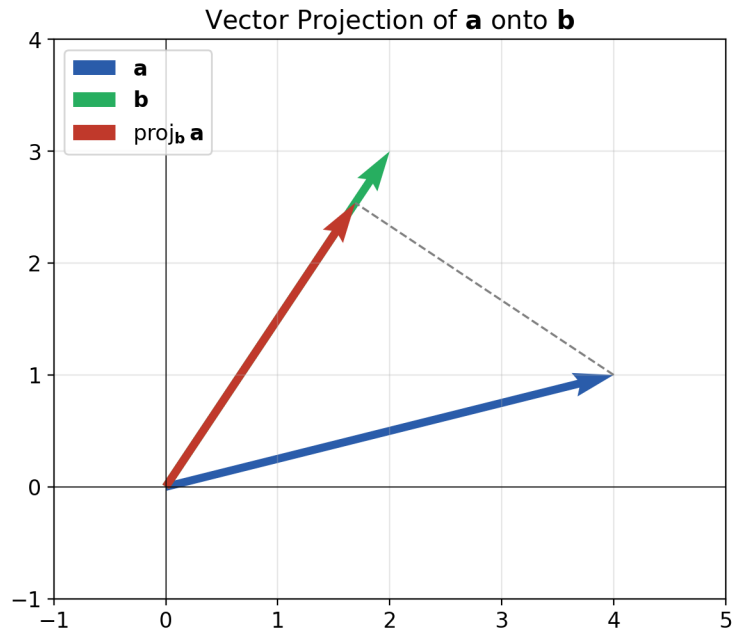


Figure 1.1: Projecting  $\mathbf{a}$  onto  $\mathbf{b}$ : the red vector is the component of  $\mathbf{a}$  that lies along  $\mathbf{b}$ .

Eigenvectors and eigenvalues reveal the "natural axes" of a linear transformation. This is the foundation of Principal Component Analysis (PCA): the eigenvectors of a data's covariance matrix point along the directions of maximum variance, and their eigenvalues tell you how much variance lies along each direction.

**Example 1.1** (PCA in one line). *Given centered data matrix  $X$ , PCA finds the eigenvectors of the covariance matrix  $\Sigma = \frac{1}{n}X^\top X$ . Projecting  $X$  onto the top- $k$  eigenvectors gives the best  $k$ -dimensional linear approximation of the data (in terms of preserved variance).*

## 1.4 Why This Matters for ML

- **Linear/logistic regression:** predictions are  $\mathbf{w}^\top \mathbf{x} + b$ .
- **Neural networks:** each layer is  $A\mathbf{x} + \mathbf{b}$  followed by an activation.
- **PCA / dimensionality reduction:** relies entirely on eigendecomposition.
- **Word embeddings, attention:** dot products measure similarity between vectors.

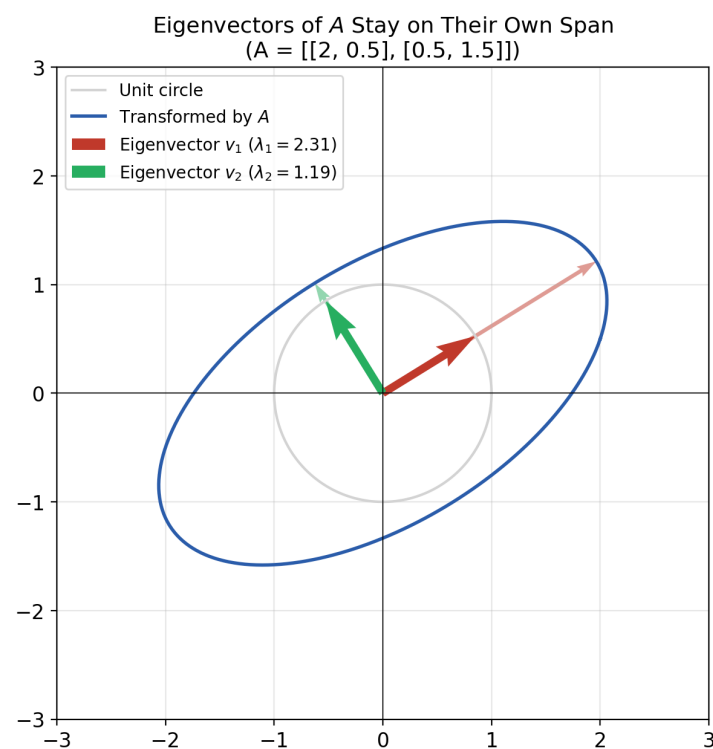


Figure 1.2: The unit circle transformed by  $A$  becomes an ellipse. Eigenvectors are the only directions that don't rotate — they only scale by their eigenvalue  $\lambda$ .

## Chapter 2

# Probability and Statistics

Machine learning models make predictions under uncertainty, and probability theory is how that uncertainty is formalized and reasoned about.

### 2.1 Random Variables and Distributions

A random variable  $X$  maps outcomes to numbers, and a probability density function (PDF)  $p(x)$  describes how likely each value is. The most important distribution in ML is the Gaussian (normal) distribution:

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

where  $\mu$  is the mean and  $\sigma^2$  is the variance.

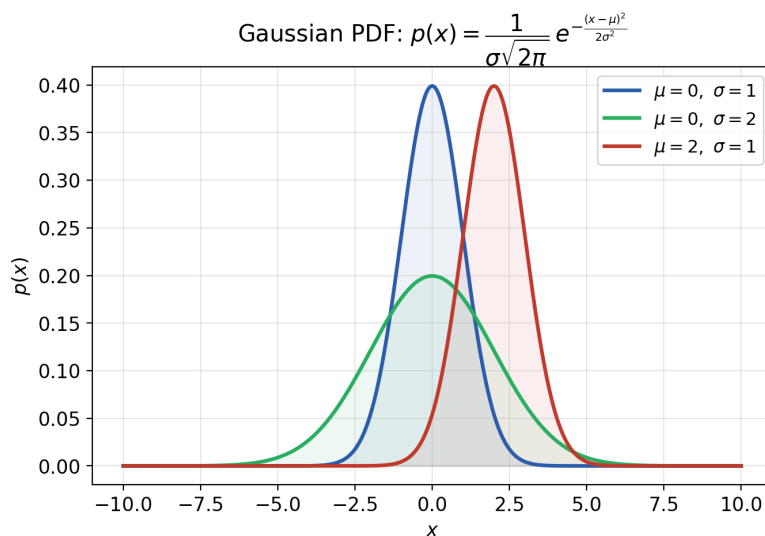


Figure 2.1: Gaussian PDFs for different means and variances. A larger  $\sigma$  spreads the distribution out;  $\mu$  shifts its center.

### 2.2 Bayes' Theorem

**Definition 2.1** (Bayes' Theorem).

$$P(A | B) = \frac{P(B | A) P(A)}{P(B)}$$

In ML terms, if  $\theta$  represents model parameters and  $D$  represents observed data:

$$\underbrace{P(\theta | D)}_{\text{posterior}} = \frac{\overbrace{P(D | \theta)}^{\text{likelihood}} \overbrace{P(\theta)}^{\text{prior}}}{\underbrace{P(D)}_{\text{evidence}}}$$

This is the mathematical basis of Bayesian machine learning: updating a prior belief about parameters using observed data to get a posterior belief.

## 2.3 Maximum Likelihood Estimation

**Definition 2.2** (Likelihood). *Given data  $x_1, \dots, x_n$  drawn i.i.d. from a distribution with unknown parameter  $\theta$ , the likelihood is*

$$\mathcal{L}(\theta) = \prod_{i=1}^n p(x_i | \theta).$$

Maximum Likelihood Estimation (MLE) chooses the parameter  $\hat{\theta}$  that makes the observed data most probable:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} \mathcal{L}(\theta) = \arg \max_{\theta} \sum_{i=1}^n \log p(x_i | \theta)$$

(We maximize the *log*-likelihood instead, since sums are easier to differentiate than products, and log is monotonic so the maximizer is unchanged.)

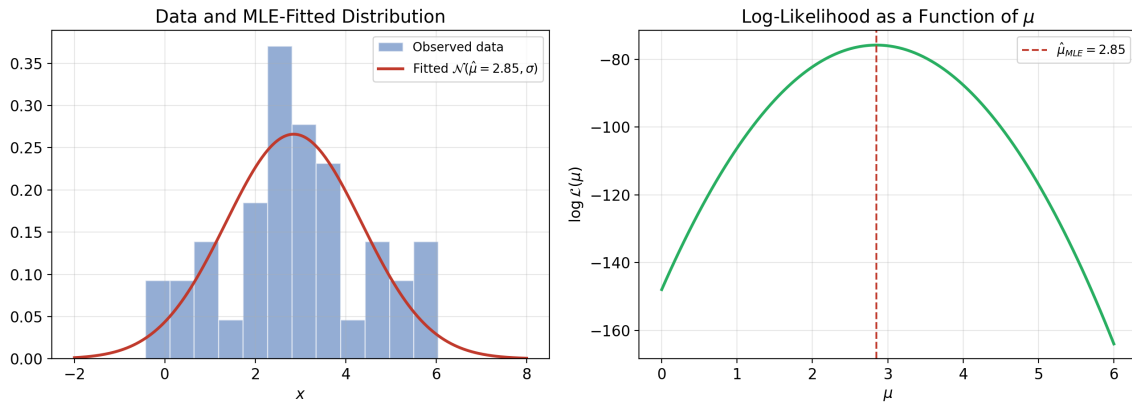


Figure 2.2: Left: data sampled from a Gaussian, with the MLE-fitted distribution overlaid. Right: the log-likelihood as a function of  $\mu$  — it's maximized exactly at the sample mean.

**Key Idea.** *Minimizing mean-squared-error loss in linear regression is equivalent to maximum likelihood estimation under the assumption that the residuals are Gaussian-distributed. This is why MSE shows up everywhere: it's not an arbitrary choice, it falls directly out of a Gaussian noise assumption.*

## 2.4 Cross-Entropy and KL Divergence

For classification, the cross-entropy between true labels  $y$  and predicted probabilities  $\hat{y}$  is

$$H(y, \hat{y}) = - \sum_i y_i \log \hat{y}_i,$$

which is exactly the negative log-likelihood of the correct class under the model's predicted distribution — this is why cross-entropy loss is the standard loss function for classifiers.

## Chapter 3

# Calculus and Optimization

Training a model means finding parameters that minimize a loss function. Calculus tells us which direction to move in; optimization algorithms tell us how to get there efficiently.

### 3.1 Derivatives and Gradients

**Definition 3.1** (Gradient). *For a scalar function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ , the gradient is the vector of partial derivatives*

$$\nabla f(\mathbf{x}) = \left( \frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right).$$

The gradient points in the direction of *steepest ascent* of  $f$ . To minimize a loss function, we move in the direction of  $-\nabla f$ .

### 3.2 The Chain Rule

**Definition 3.2** (Chain Rule). *If  $z = f(y)$  and  $y = g(x)$ , then*

$$\frac{dz}{dx} = \frac{dz}{dy} \cdot \frac{dy}{dx}.$$

The chain rule is the mathematical backbone of backpropagation: a neural network is a composition of many functions (layers), and computing the gradient of the loss with respect to early-layer weights requires multiplying together the derivatives of every layer in between.

### 3.3 Gradient Descent

Given a loss function  $J(\theta)$ , gradient descent iteratively updates parameters:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla J(\theta^{(t)})$$

where  $\eta$  is the **learning rate**, controlling step size.

**Key Idea.** *The learning rate  $\eta$  trades off speed and stability. Too small and training crawls; too large and updates overshoot the minimum or diverge entirely. Elongated, narrow bowls (as in the figure above, where one direction is steeper than the other) are exactly why adaptive optimizers like Adam and RMSProp were developed — they rescale step sizes per parameter.*

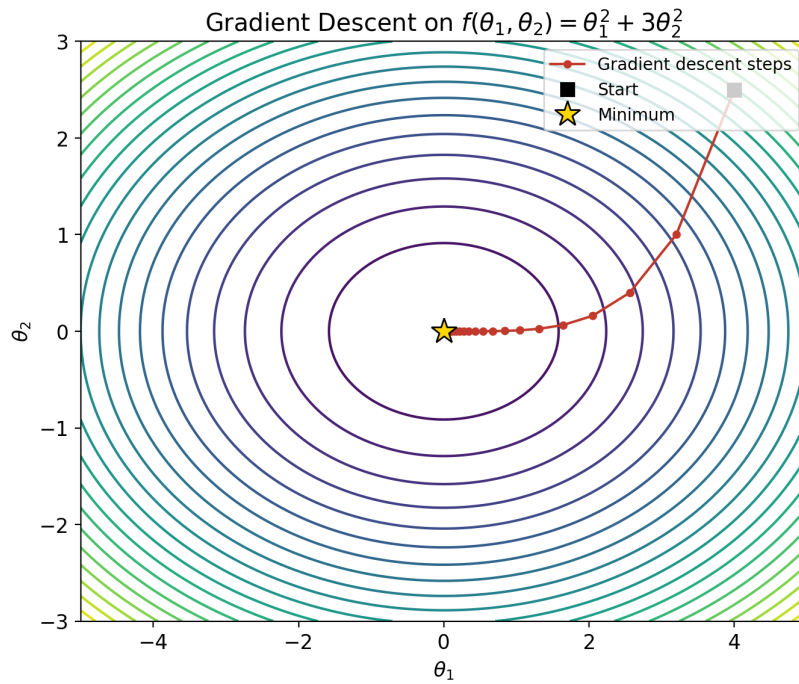


Figure 3.1: Gradient descent path on a convex bowl-shaped loss  $J(\theta_1, \theta_2) = \theta_1^2 + 3\theta_2^2$ . Each step moves opposite the gradient, converging toward the minimum.

### Loss Surface and Descent Trajectory

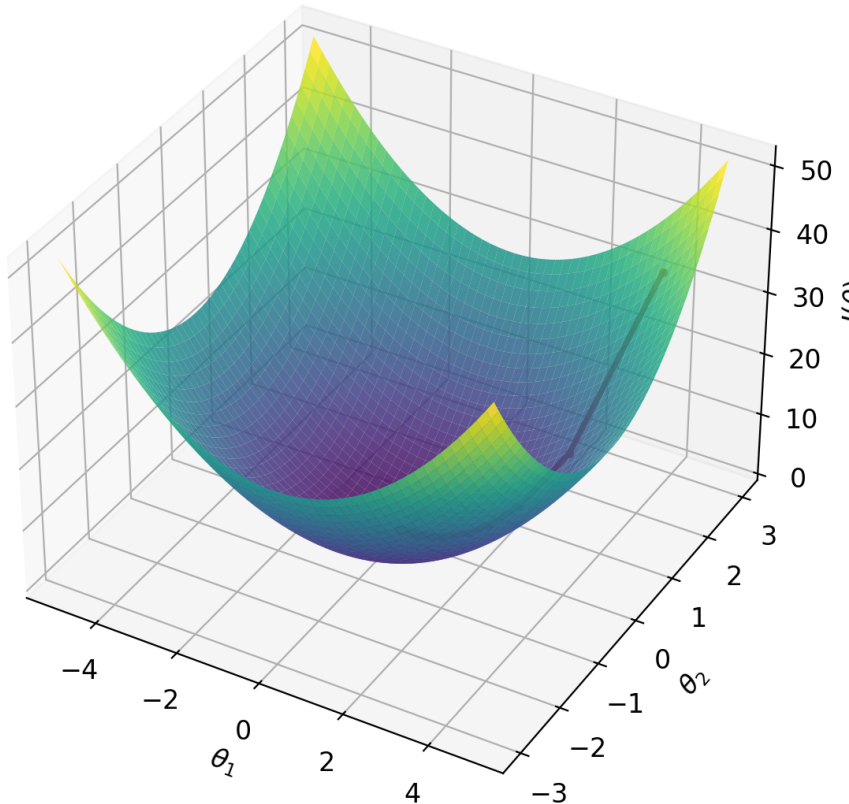


Figure 3.2: The same loss function viewed as a 3D surface, with the descent trajectory rolling downhill toward the minimum.

### 3.4 Variants of Gradient Descent

- **Batch GD:** uses the entire dataset per update — accurate but slow.
- **Stochastic GD (SGD):** uses one example per update — noisy but fast, and the noise can help escape shallow local minima.
- **Mini-batch GD:** the standard compromise, updating on small batches (e.g. 32–256 examples).
- **Momentum:** accumulates a moving average of past gradients to smooth out the path and accelerate convergence.
- **Adam:** combines momentum with a per-parameter adaptive learning rate, and is the default optimizer in most modern deep learning.

### 3.5 Convexity

**Definition 3.3** (Convex Function).  *$f$  is convex if for all  $\mathbf{x}, \mathbf{y}$  and  $t \in [0, 1]$ ,*

$$f(t\mathbf{x} + (1 - t)\mathbf{y}) \leq tf(\mathbf{x}) + (1 - t)f(\mathbf{y}).$$

For convex loss functions (like linear regression’s MSE), gradient descent is guaranteed to converge to the *global* minimum. Neural network loss surfaces are generally *non-convex*, which is why training deep networks well requires careful initialization, learning-rate schedules, and architectural choices — there’s no guarantee of finding the global optimum.

## Chapter 4

# Regression and Classification

Linear and logistic regression are the simplest ML models, but they contain — in miniature — every mathematical idea used by deeper models: a parametric function, a loss function, and gradient-based fitting.

### 4.1 Linear Regression

The model predicts a continuous target as a linear combination of features:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

**Definition 4.1** (Mean Squared Error).

$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Because  $J$  is convex and quadratic in  $\mathbf{w}$ , it has a closed-form solution — the **normal equation**:

$$\mathbf{w}^\star = (X^\top X)^{-1} X^\top \mathbf{y}$$

In practice, gradient descent is often used instead, especially when  $X^\top X$  is large or ill-conditioned to invert directly.

### 4.2 Logistic Regression

For binary classification, we squash the linear output through a sigmoid to get a probability:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \in (0, 1)$$

**Definition 4.2** (Binary Cross-Entropy Loss).

$$J(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[ y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Unlike linear regression, logistic regression has no closed-form solution —  $J$  is convex, but nonlinear, so it's fit with gradient descent. The gradient turns out to have an elegantly simple form:

$$\nabla_{\mathbf{w}} J = \frac{1}{n} X^\top (\hat{\mathbf{y}} - \mathbf{y})$$

which looks just like the linear regression gradient, with  $\hat{y}$  now being the sigmoid output instead of the raw linear prediction.

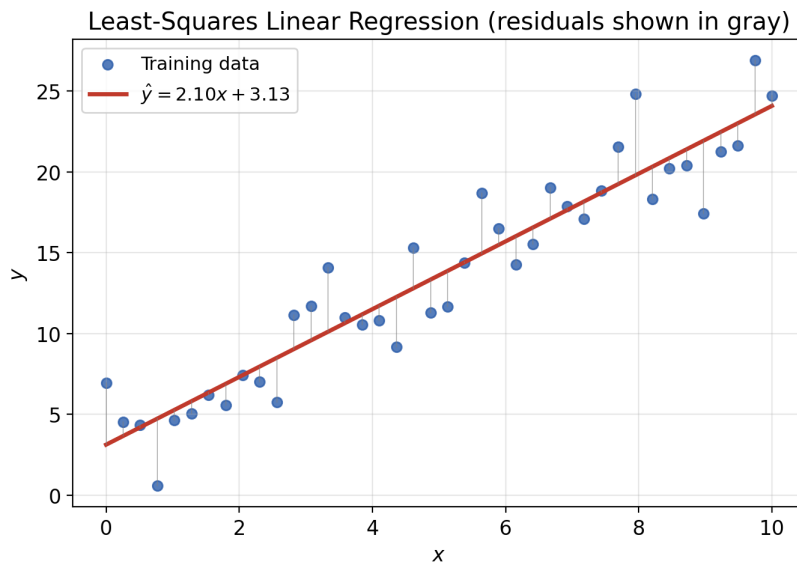


Figure 4.1: Least-squares fit to noisy data. Gray lines show residuals — the vertical distances MSE penalizes.

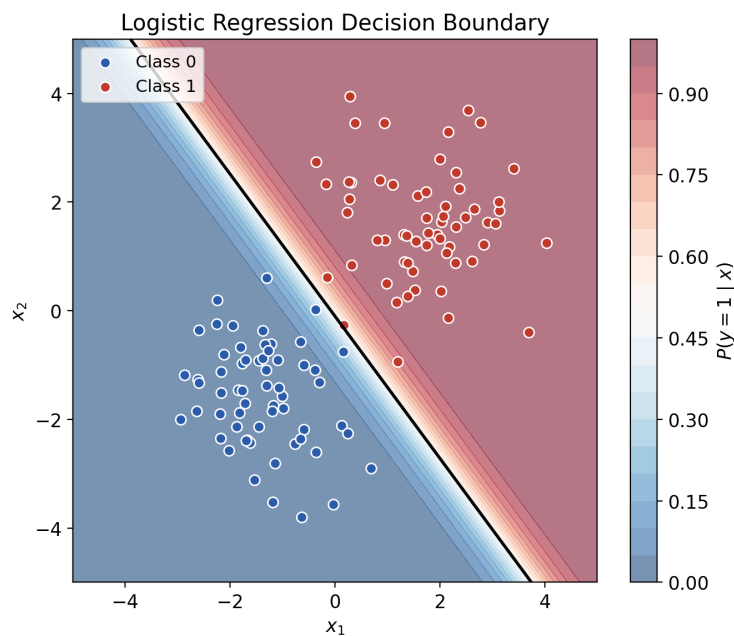


Figure 4.2: Logistic regression fit to two Gaussian-distributed classes. Color shows predicted probability  $P(y = 1 | x)$ ; the black curve is the decision boundary at  $P = 0.5$ .

### 4.3 Regularization

To prevent overfitting, a penalty on the weights is added to the loss:

$$J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \underbrace{\|\mathbf{w}\|_2^2}_{\text{L2 (Ridge)}} \quad \text{or} \quad J_{\text{reg}}(\mathbf{w}) = J(\mathbf{w}) + \lambda \underbrace{\|\mathbf{w}\|_1}_{\text{L1 (Lasso)}}$$

**Key Idea.** *L2 regularization shrinks weights smoothly toward zero (but rarely exactly zero), while L1 regularization tends to push many weights to exactly zero — producing sparse models that effectively perform feature selection. This difference comes directly from the geometry of the  $\ell_1$  vs.  $\ell_2$  norm balls.*

## Chapter 5

# Neural Networks and Backpropagation

A neural network is a stack of linear transformations separated by nonlinear activation functions. Everything from Chapters 1–4 — matrix multiplication, the chain rule, gradient descent, cross-entropy loss — comes together here.

### 5.1 The Forward Pass

For a layer  $l$ , the pre-activation and activation are

$$\mathbf{z}^{(l)} = W^{(l)} \mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma(\mathbf{z}^{(l)}),$$

where  $\mathbf{a}^{(0)} = \mathbf{x}$  is the input and  $\sigma$  is a nonlinear activation function applied elementwise.

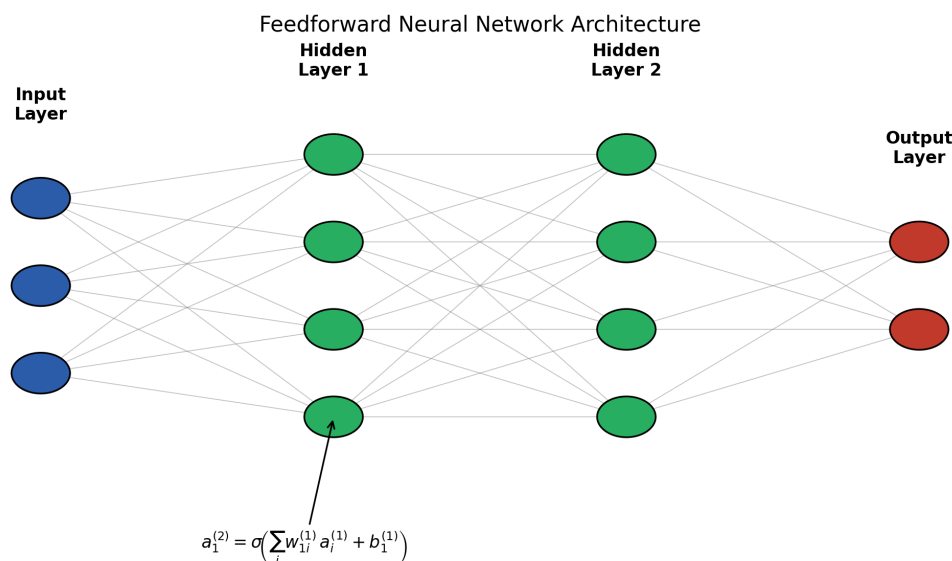


Figure 5.1: A feedforward network: each neuron computes a weighted sum of the previous layer’s activations, plus a bias, passed through  $\sigma$ .

### 5.2 Activation Functions

Without a nonlinearity, stacking linear layers would collapse into a single linear transformation —  $\sigma$  is what gives networks the power to approximate complex, non-linear functions.

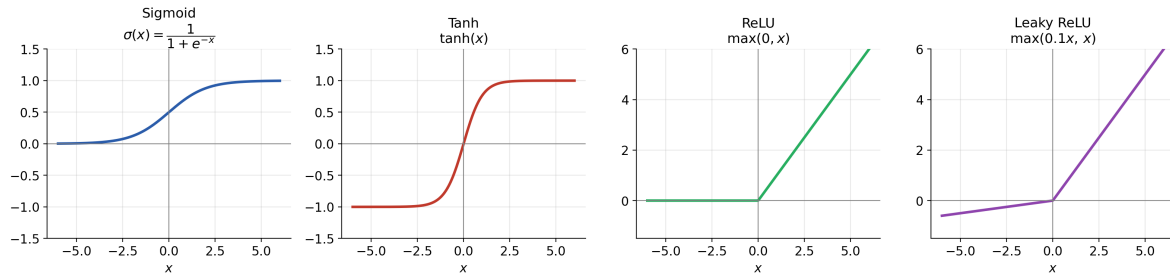


Figure 5.2: Common activation functions. Sigmoid and tanh saturate for large  $|x|$  (causing vanishing gradients); ReLU and its variants avoid this for positive inputs and are the default choice in modern deep networks.

**Key Idea.** *ReLU's simplicity ( $\max(0, x)$ ) is exactly why it works so well: its gradient is either 0 or 1, so it doesn't shrink gradients during backpropagation the way sigmoid/tanh do when saturated. Leaky ReLU further fixes the "dying ReLU" problem by keeping a small gradient for  $x < 0$ .*

### 5.3 The Loss Function

For classification with  $K$  classes, the network outputs are passed through a softmax to form a probability distribution:

$$\hat{y}_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}},$$

and trained with cross-entropy loss, exactly as in Chapter 4.

### 5.4 Backpropagation

Backpropagation is the chain rule (Chapter 3), applied systematically from the output layer back to the input layer, to compute  $\partial J / \partial W^{(l)}$  for every layer.

Define the **error signal** at layer  $l$  as  $\delta^{(l)} = \frac{\partial J}{\partial \mathbf{z}^{(l)}}$ . Then:

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} J \odot \sigma'(\mathbf{z}^{(L)}) \quad (\text{output layer})$$

$$\delta^{(l)} = \left(W^{(l+1)}\right)^\top \delta^{(l+1)} \odot \sigma'(\mathbf{z}^{(l)}) \quad (\text{hidden layers, propagated backward})$$

$$\frac{\partial J}{\partial W^{(l)}} = \delta^{(l)} \left(\mathbf{a}^{(l-1)}\right)^\top \quad (\text{weight gradient})$$

$$\frac{\partial J}{\partial \mathbf{b}^{(l)}} = \delta^{(l)} \quad (\text{bias gradient})$$

where  $\odot$  denotes elementwise multiplication.

**Key Idea.** *Backpropagation is just an efficient bookkeeping trick for the chain rule: instead of recomputing derivatives from scratch for every weight, error signals  $\delta^{(l)}$  are computed once per layer and reused, making training cost roughly proportional to a single forward pass, not exponential in network depth.*

Once every  $\partial J / \partial W^{(l)}$  is known, ordinary gradient descent (Chapter 3) updates each weight matrix:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial J}{\partial W^{(l)}}.$$

## 5.5 Putting It All Together

1. **Linear algebra** (Ch. 1): every layer is a matrix multiplication.
2. **Probability** (Ch. 2): the loss is a negative log-likelihood (cross-entropy).
3. **Calculus** (Ch. 3): gradients are computed via the chain rule and used to descend the loss surface.
4. **Regression** (Ch. 4): a single neuron with a sigmoid *is* logistic regression — a neural network is many of these, composed and stacked.

This is the complete mathematical loop that trains every modern deep learning model, from a two-layer classifier to a large language model: forward pass, loss, backward pass, update — repeated millions of times.